

Mixture-of-Collaboration: Expert Collaboration via Learned Mediators in Sparse Transformer Architectures

Technical Report

Francisco Antonio
Auren Research, Independent

April 2026

Abstract

Mixture-of-Experts (MoE) Transformers scale parameter count while keeping per-token compute bounded, but standard top- k routing still treats selected experts as conditionally independent paths whose outputs are merged only at the end. This report presents *Mixture-of-Collaboration* (MoC), a sparse decoder-only Transformer that augments MoE with explicit expert interaction through a learned mediator. MoC combines three mechanisms: (1) mediator-based collaboration over routed experts with $\mathcal{O}(K)$ token complexity, avoiding pairwise $\mathcal{O}(E^2)$ expert communication; (2) Iterative Reasoning Layers (IRL), where each expert performs weight-shared multi-step refinement; and (3) adaptive compute gates that learn, per token, how many reasoning steps and collaboration rounds to allocate.

We now report two stages of evidence. First, in a controlled small-scale ablation at approximately 64M parameters, MoC improves validation perplexity from 59.33 for a standard MoE baseline to 55.67 with fixed collaboration, while a later adaptive pilot reaches 56.02 under an earlier configuration branch. Second, at 498.8M total parameters trained on approximately 9.83B tokens, MoC improves validation perplexity from 20.70 to 20.40 against a matched MoE baseline with identical total and active parameters. At that larger scale, MoC also improves routing quality, reducing the maximum per-layer expert-usage Gini coefficient from 0.283 to 0.153 with no dead experts. These gains come with a substantial efficiency cost—roughly $2\times$ wall-clock time in the current implementation—which clarifies the next optimization target. We report exact run settings, throughput, routing diagnostics, and scaling trends for reproducibility.

1 Introduction

Sparse Transformers based on MoE routing can substantially increase total parameter capacity without making every token pay full dense-compute cost [3, 5, 8, 9]. In standard top- k MoE, a router selects experts and the selected experts process token states independently before a weighted merge. This design is efficient, but it also constrains information exchange among experts to a single late aggregation point.

An immediate idea is to let experts communicate directly. However, many communication-heavy variants introduce pairwise interactions whose cost scales with the number of experts, making them difficult to scale at high expert counts. MoC instead adopts a learned mediator hub: experts communicate through a shared latent state, so per-token collaboration remains linear in selected experts ($\mathcal{O}(K)$), not quadratic in total experts.

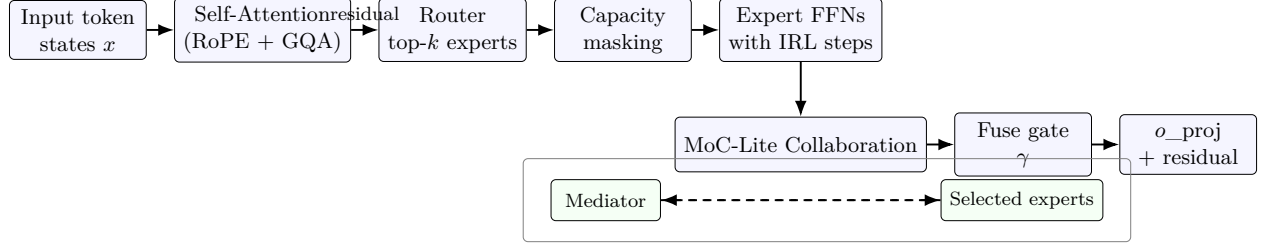


Figure 1: MoC block flow. Tokens are routed to top- k experts, filtered by capacity, processed with IRL, then refined by mediator-based collaboration before gated fusion and output projection.

A second issue is compute allocation. Fixed-depth processing assumes all tokens require equal effort, but tokens in language modeling vary widely in difficulty. A uniform budget may over-compute trivial tokens and under-compute difficult ones. MoC therefore incorporates adaptive gates for both intra-expert reasoning depth and inter-expert collaboration rounds.

Contributions.

- We introduce **MoC**, a sparse MoE architecture with mediator-based expert collaboration at linear cost in top- k experts.
- We integrate **Iterative Reasoning Layers (IRL)** inside experts via weight-shared multi-step refinement.
- We add **adaptive compute gating** for both reasoning depth and collaboration rounds.
- We provide a **controlled ablation** at small scale and a **matched large-scale comparison** at 500M parameters / 10B tokens.

This document is a technical report (not a peer-reviewed paper) intended to record architecture details, updated experimental evidence, and current scaling limits with reproducible settings.

2 Architecture

2.1 Overview

The implementation in `model_moc.py` is a decoder-only Transformer with Rotary Position Embeddings (RoPE), grouped-query attention (GQA), RMSNorm, and SwiGLU-style FFNs. Each transformer block applies:

$$\text{Attention} \rightarrow \text{Router} \rightarrow \text{Expert Dispatch} \rightarrow \text{MoC Collaboration} \rightarrow \text{Fuse/Project} \rightarrow \text{Residual}.$$

When `n_experts` is `None`, blocks use a dense FFN path. Otherwise, each block uses `MoCTopKExperts` for routing, capacity filtering, per-expert IRL computation, and optional collaboration/fusion.

2.2 Router and Expert Dispatch

Given flattened token states $x \in \mathbb{R}^{N \times d}$, routing logits are computed in FP32:

$$\ell = xW_{\text{gate}}^{\top}, \quad p = \text{softmax}(\ell). \quad (1)$$

During training, Gaussian router noise (std 0.02 in experiments) perturbs logits for exploration. Top- k experts are selected per token and top- k probabilities are normalized for weighted aggregation. Capacity is enforced via a vectorized priority-based keep/drop policy:

- flatten all token-expert slots ($N \times K$),
- stable sort by descending routing priority,
- regroup by expert,
- keep up to per-expert capacity, drop the rest.

The code sets capacity from

$$C = 2^{\lceil \log_2(\lceil \text{capacity_factor} \cdot (NK/E) \rceil) \rceil}, \quad (2)$$

clipped to at most NK . Experiments use `capacity_factor=1.5`.

Auxiliary router regularization is:

$$\mathcal{L}_{\text{aux}} = \lambda_{\text{bal}} \mathcal{L}_{\text{bal}} + \lambda_z \mathcal{L}_z + \lambda_{\text{drop}} \mathcal{L}_{\text{drop}}, \quad (3)$$

where $\lambda_{\text{bal}} = 0.01$, $\lambda_z = 0.001$, and $\lambda_{\text{drop}} = 0.001$ in MoE/MoC runs. The implementation computes:

$$\mathcal{L}_{\text{bal}} = E \sum_{e=1}^E \bar{p}_e \bar{f}_e, \quad (4)$$

$$\mathcal{L}_z = \mathbb{E}[\log \sum_e \exp(\ell_e)]^2, \quad (5)$$

$$\mathcal{L}_{\text{drop}} = \text{drop fraction}. \quad (6)$$

2.3 Iterative Reasoning Layers (IRL)

Each expert is a weight-shared iterative FFN:

$$h^{(0)} = x, \quad h^{(t+1)} = h^{(t)} + \alpha \text{FFN}(\text{RMSNorm}(h^{(t)} + x)), \quad (7)$$

for $t = 0, \dots, N_{\text{max}} - 1$, with $\alpha = 1/\sqrt{N_{\text{max}}}$ for stability. The FFN uses SwiGLU structure and reuses the same parameters at every step.

This recurrent refinement is related to equilibrium-style views of deep computation [1]: although MoC runs a finite number of iterations, the shared-weight formulation encourages fixed-point-like behavior while keeping parameter growth controlled.

2.4 MoC-Lite Collaboration

After expert computation, MoC-Lite refines token representation through a learned mediator state. For each token, let selected expert states be $\{e_i\}_{i=1}^K$ and weighted MoE merge be $\tilde{e} = \sum_i w_i e_i$.

Mediator initialization is:

$$m_0 = m_{\text{learned}} + W_{\text{ctx}} x \quad (8)$$

(or a fallback from $\tilde{e}$ if learned mediator is disabled).

One collaboration round computes:

$$q = W_q \text{Norm}(m), \quad (9)$$

$$k_i = W_k \text{Norm}(e_i), \quad v_i = W_v \text{Norm}(e_i), \quad (10)$$

$$a_i = \text{softmax}_i \left(\frac{q^\top k_i}{\sqrt{d}} \right), \quad (11)$$

$$u = \sum_i a_i v_i, \quad (12)$$

$$m' = m + \text{MLP}(\text{Norm}(m) + u). \quad (13)$$

If expert feedback is enabled (enabled in experiments), mediator-to-expert feedback is gated per expert:

$$g_i = \sigma(w_e^\top e_i + w_m^\top m'), \quad e'_i = e_i + g_i W_{m \rightarrow e} m'. \quad (14)$$

Final fusion between mediator and weighted expert sum uses an input-conditioned gate:

$$\gamma = \sigma(W_f x + b), \quad y = \gamma m + (1 - \gamma) \sum_i w_i e'_i. \quad (15)$$

This hub topology keeps collaboration complexity linear in selected experts ($\mathcal{O}(K)$ per token) instead of all-expert pairwise interaction.

2.5 Adaptive Compute Gating

MoC includes two independent adaptive gates:

- **Reasoning gate** for IRL depth (0 to $N_{\max}$ steps).
- **Collaboration gate** for mediator rounds (0 to $C_{\max}$ rounds).

Each gate maps token state to logits over $\{0, 1, \dots, S\}$ and applies temperature-scaled softmax. In **soft** mode (used in experiments), activity weights are cumulative tail probabilities, enabling compile-friendly dense execution:

$$\text{activity}_s = \sum_{j=s}^S P(\text{steps} = j). \quad (16)$$

In **hard** mode, the model selects an argmax step count and executes sparse per-token control flow. Expected-step penalties encourage efficiency:

$$\mathcal{L}_{\text{step}} = \lambda_{\text{step}} \frac{\mathbb{E}[\text{steps}]}{S}, \quad (17)$$

with weight 0.001 for both reasoning and collaboration in MoC-vNext. Gate temperature is 1.0, gate noise std is 0.02.

Table 1: Experiment configurations summarized by scale.

Run	Routing	IRL	Collaboration	Hardware	Total params	Active/token	Batch $\times$ GAS
a-vanilla	dense FFN	1 fixed step	off	A10 24GB	35,789,952	35,789,952	32×1
b-moe	$E = 4$, top-2, cap=1.25	1 fixed step	off	A10 24GB	64,138,398	49,978,014	16×2
c-moc-v1	$E = 4$, top-2, cap=1.5	2 fixed steps	mediator on, feedback on, 2 rounds	A10 24GB	64,131,468	49,971,084	16×2
e-moc-v2-L40S	$E = 4$, top-2, cap=1.5	2 fixed steps	mediator on, feedback on, 2 rounds	L40S 48GB	64,131,468	49,971,084	16×2
f-moc-v2-scaled-200M	$E = 8$, top-2, cap=1.5	2 fixed steps	mediator on, feedback on, 2 rounds	L40S 48GB	197,200,000	93,400,000	16×4
moe-500m-10b-mi300x	$E = 8$, top-2, cap=1.5	2 fixed steps	off	MI300X 192GB	498,800,000	215,600,000	64×2
moc-500m-10b-mi300x	$E = 8$, top-2, cap=1.5	2 fixed steps	mediator on, feedback on, 2 rounds	MI300X 192GB	498,800,000	215,600,000	64×2

3 Experimental Setup

3.1 Model Configurations

The updated report covers three experimental regimes: (i) a controlled 64M-scale ablation on A10, (ii) a 200M MoC scaling run on L40S under the same token budget, and (iii) a matched 500M MoE/MoC comparison on MI300X trained for 10B tokens. All runs use the same tokenizer (65,536 vocabulary items) and a sequence length of 512.

All sparse runs share AdamW with betas (0.9, 0.95), weight decay 0.1, gradient clipping at 1.0, base learning rate 3×10^{-4} , cosine decay, router noise standard deviation 0.02, and the usual MoE auxiliary losses (balance 0.01, z -loss 0.001, drop penalty 0.001). The earlier adaptive pilot, `d-moc-vnext`, remains useful as a design probe but is no longer the strongest empirical result in this report, so we treat it as supplementary rather than primary evidence.

3.2 Training Setup

- **Sequence length:** 512 for every run.
- **Small-scale budget:** 20,000 steps and approximately 327M tokens ($20,000 \times 16 \times 2 \times 512$ or equivalent effective batch).
- **200M scaling run:** 10,000 steps and approximately 327M tokens ($10,000 \times 16 \times 4 \times 512$).
- **500M comparison:** 150,000 steps and approximately 9.83B tokens ($150,000 \times 64 \times 2 \times 512$).
- **Warmup:** 500 steps for the small-scale runs and 1,000 steps for the 500M runs.
- **Precision:** bf16 mixed precision.
- **Seed:** 1337 for all reported checkpoints.

3.3 Hardware

- NVIDIA A10 24GB for the baseline 64M ablations.
- NVIDIA L40S 48GB for the 64M rerun and the 200M scaling test.
- AMD Instinct MI300X 192GB for the matched 500M / 10B-token comparison.
- All results are single-GPU runs without distributed training.

Table 2: Updated results across scales.

Run	Params (total $\rightarrow$ active)	Steps	Tokens	Val PPL	Val loss	tok/s
a-vanilla	35.8M $\rightarrow$ 35.8M	20k	$\sim$ 327M	67.68	4.2147	71.4k
b-moe	64.1M $\rightarrow$ 50.0M	20k	$\sim$ 327M	59.33	4.0933	47.1k
c-moc-v1	64.1M $\rightarrow$ 50.0M	20k	$\sim$ 327M	55.67	4.0298	23.6k
e-moc-v2-L40S	64.1M $\rightarrow$ 50.0M	20k	$\sim$ 327M	55.42	4.0253	48.1k
f-moc-v2-scaled-200M	197.2M $\rightarrow$ 93.4M	10k	$\sim$ 327M	45.73	3.8335	26.9k
moe-500m-10b-mi300x	498.8M $\rightarrow$ 215.6M	150k	$\sim$ 9.83B	20.70	3.0408	95.8k
moc-500m-10b-mi300x	498.8M $\rightarrow$ 215.6M	150k	$\sim$ 9.83B	20.40	3.0155	53.7k

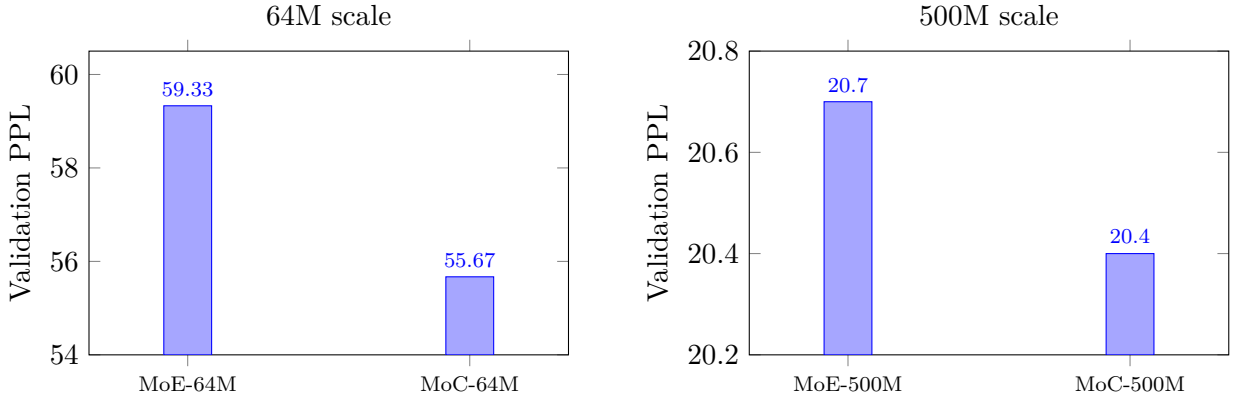


Figure 2: Perplexity comparisons with per-scale axes. Left: at 64M, MoC improves over the matched MoE baseline from 59.33 to 55.67. Right: at 500M trained on 9.83B tokens, MoC improves from 20.70 to 20.40. Routing quality is also better at 500M, with maximum Gini improving from 0.283 to 0.153.

3.4 Evaluation

We report validation loss, validation perplexity, throughput (tokens/s), wall-clock time, and routing-health metrics when available. For the 500M runs, routing diagnostics include dead-expert counts, layer-wise entropy, maximum Gini coefficient, layer agreement, and drop rate.

4 Results

4.1 Main Results

Table 2 consolidates the updated experimental record.

The updated evidence changes the paper’s center of gravity. At 64M, MoC clearly outperforms the MoE baseline under the same token budget, improving perplexity from 59.33 to 55.67. At 500M and 9.83B tokens, MoC still retains an advantage under a matched architecture and matched active parameter budget, reducing validation perplexity from 20.7038 to 20.3991, a 2.47% relative gain. The larger result is especially important because it survives training on roughly $30\times$ more tokens (9.83B versus 327M) rather than appearing only in the short pilot regime.

Table 3: Incremental ablation at 64M scale.

Step	Added component(s)	PPL	Incremental Δ
1	Dense baseline (single FFN, no routing)	67.68	—
2	Add top-2 expert routing ($E = 4$), independent experts, weighted merge	59.33	-12.3%
3	Add mediator collaboration and two fixed reasoning/collaboration steps	55.67	-6.2%
4	Re-run MoC v2 on L40S with essentially unchanged architecture	55.42	-0.4%

Table 4: Scaling observations from the updated runs.

Run	Total params	Active params	Val PPL	Relative change
e-moc-v2-L40S	64.1M	50.0M	55.42	—
f-moc-v2-scaled-200M	197.2M	93.4M	45.73	-17.5% vs 64M MoC
moc-500m-10b-mi300x	498.8M	215.6M	20.40	-55.4% vs 200M MoC

4.2 Ablation and Scaling Analysis

The controlled 64M ablation remains useful because it isolates which ingredients matter most before scale effects dominate. Table 3 updates those numbers from the latest experiment log.

The 64M results show that expert routing provides the largest single gain, but explicit collaboration remains meaningful: MoC-v1 improves by 6.2% relative to the matched MoE baseline at the same scale. The L40S rerun is numerically close to the A10 MoC result, which suggests the gain is architectural rather than hardware-specific.

The scaling trend is encouraging but should be interpreted carefully because both model size and training budget change. Increasing MoC from 64M to 197M under the same token budget lowers perplexity by 17.5%. Moving to 498.8M with a 10B-token budget lowers perplexity much further, indicating that MoC remains competitive well beyond the original pilot regime.

4.3 Efficiency and Routing Health

The principal trade-off is efficiency. At 500M scale, MoC improves perplexity and routing quality, but its current implementation is substantially slower: 53.7k tokens/s versus 95.8k for the matched MoE baseline, and 43h25m wall time versus 21h10m. This strongly suggests that collaboration overhead is algorithmically worthwhile but systems-limited in the current code path.

Routing diagnostics at 500M are favorable:

- no dead experts in any of the 10 layers for either MoE or MoC
- maximum Gini coefficient improves from 0.283 (MoE) to 0.153 (MoC)
- mean router entropy is higher for MoC (roughly 1.82–1.90 nats) than for MoE (roughly 1.73–1.80)
- drop rate is effectively zero, with only layer 9 in the 500M MoC run reaching 0.042%

These routing statistics strengthen the empirical case for mediator-based collaboration: MoC is not merely lowering perplexity, it is doing so while using experts more evenly.

5 Related Work

MoC is situated at the intersection of four lines of work.

Sparse MoE scaling. Prior work established sparse expert routing as an effective scaling strategy, including Switch Transformer [5], GShard [9], Mixtral [8], and DeepSeek-MoE [3]. These methods primarily focus on routing and specialization with expert outputs merged after independent processing.

Expert communication. Soft MoE [10] replaces discrete top- k dispatch with fully differentiable soft assignment, where each expert receives weighted combinations of tokens. HyperMoE [12] supplements expert knowledge via hypernetwork-generated transfers from unselected experts. MoC occupies a different design point: it preserves sparse top- k routing and adds post-selection collaboration among chosen experts via a shared mediator hub, yielding linear complexity in top- k experts rather than dense interaction.

Adaptive compute. Adaptive-depth and adaptive-halting ideas appear in Universal Transformer [4], PonderNet [2], and CALM [11]. MoC adopts this principle inside sparse expert layers and collaboration modules.

Iterative/implicit reasoning. Weight-shared iterative refinement connects to deep equilibrium perspectives [1]. Emerging latent-reasoning directions such as Coconut [7] and pause-token delayed-compute methods [6] further motivate allocating extra internal computation before committing outputs.

To our knowledge, MoC is among the first implementations combining sparse routing, explicit mediator collaboration, iterative expert refinement, and adaptive compute gating in one unified decoder architecture.

6 Limitations and Future Work

6.1 Limitations

This report intentionally emphasizes limitations:

- still based on a small number of runs per setting, with a single seed throughout
- single seed (1337), no confidence intervals
- evaluation is still limited to one dataset family and sequence length 512
- the strongest large-scale evidence is one matched 500M comparison rather than a full sweep
- expert counts above 8 and top- k settings beyond 2 remain untested
- per-expert execution still uses a Python loop (no grouped GEMM path)
- evaluation remains limited to perplexity; no downstream task suite is included yet
- fuse gate conditions only on original input x , not on collaboration output

- MoC currently incurs a substantial throughput penalty relative to plain MoE
- long-context behavior remains untested

6.2 Future Work

Priority next steps:

- optimize collaboration kernels and replace Python-side expert loops with grouped GEMM or fused kernels
- repeat the 500M comparison across multiple seeds and data-order variations
- extend scale studies to 300M, 600M, and 1B parameters
- multi-seed trials with confidence intervals
- larger expert pools (8, 16, and 32 experts)
- downstream benchmarks (MMLU, HumanEval, GSM8K)
- long-context training/evaluation (2048 and 4096)
- hybrid latent + explicit reasoning designs (IRL with CoT-like supervision)
- TPU/XLA port for cross-hardware validation
- fuse-gate variants conditioned on collaboration output

7 Conclusion

The updated evidence strengthens the core claim of this report. At small scale, MoC improves over standard MoE by enabling selected experts to collaborate through a learned mediator. At larger scale, that advantage persists: in a matched 498.8M-parameter comparison trained on 9.83B tokens, MoC reaches 20.40 validation perplexity versus 20.70 for MoE while also improving expert load balancing. The current implementation pays a clear efficiency penalty, so the most important next step is systems optimization rather than architectural rescue. In short, the new data shift MoC from an interesting pilot idea to a credible sparse-architecture direction with measurable gains and a well-defined engineering bottleneck.

8 Reproducibility Statement

- Code repository: <https://github.com/Auren-Research/lunaris> (tag v0.1.0-beta)
- Experiment logs: <https://wandb.ai/smerlylle-moon-cloud-services-/lunaris-moc-validation>
- Exact YAML configs: included in the repository for the small-scale, 200M, and 500M runs summarized here
- Hardware used: NVIDIA A10 24GB, NVIDIA L40S 48GB, and AMD Instinct MI300X 192GB

References

- [1] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL <https://arxiv.org/abs/1909.01377>.
- [2] Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. *arXiv preprint arXiv:2107.05407*, 2021. URL <https://arxiv.org/abs/2107.05407>.
- [3] Damai Dai, Chao Deng, Cheng Zhao, Rihuan Xu, Shihao Tang, Renrui Chen, Deli Zhao, Kai Yu, and Zhifang Sui. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024. URL <https://arxiv.org/abs/2401.06066>.
- [4] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.
- [5] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022. URL <https://jmlr.org/papers/v23/21-0998.html>.
- [6] Sachin Goyal, Krishnamurthy Dvijotham, Su Wang, Alexander Todorov, Daniel Korat, and Sarath Chandar. Think before you speak: Training language models with pause tokens. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=oBpHVLzkfm>.
- [7] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Coconut: Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024. URL <https://arxiv.org/abs/2412.06769>.
- [8] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. URL <https://arxiv.org/abs/2401.04088>.
- [9] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=qrwe7XHTmYb>.
- [10] Joan Puigcerver, Carlos Riquelme Ruiz, Basil Mustafa, and Neil Houlsby. From sparse to soft mixtures of experts. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=jxpsAj7ltE>.
- [11] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *arXiv preprint arXiv:2207.07061*, 2022. URL <https://arxiv.org/abs/2207.07061>.
- [12] Hao Zhao, Zihan Qiu, Huijia Wu, Zili Wang, Zhaofeng He, and Jie Fu. HypermoE: Towards better mixture of experts via transferring among experts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10605–10618, 2024. URL <https://aclanthology.org/2024.acl-long.571/>.